第 8 章 堆和 priority_queue

1. 堆的定义

（可以用 ppt 展示）

堆（heap），是一棵有着特殊性质的完全二叉树，可以用来实现优先级队列（priority queue）。

堆需要满足以下性质：

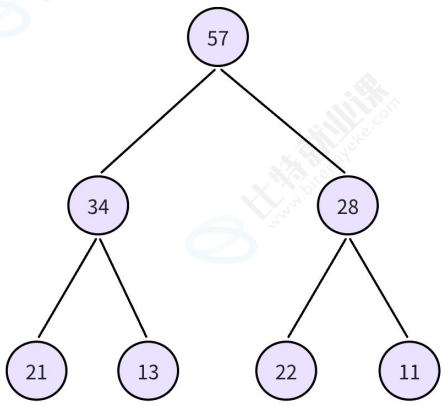
> 1. 是一棵完全二叉树；
> 2. 对于树中每个结点，如果存在子树，那么该结点的权值大于等于（或小于等于）子树中所有结点的权值。

如果根结点大于等于子树结点的权值，称为大根堆；反之，称为小根堆。

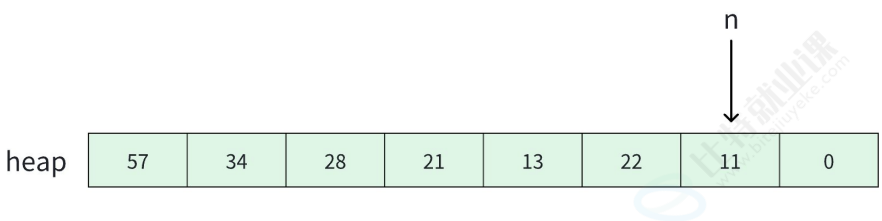
2. 堆的存储

由于堆是**一个完全二叉树**，因此**可以用一个数组来存储**。（如果不清楚的同学，可以回顾《二叉树》中的**顺序存储**部分）

> 结点下标为 i ：
>
> - 如果父存在，父下标为 $i/2$;
> - 如果左孩子存在，左孩子下标为 $i \times 2$;
> - 如果右孩子存在，右孩子下标为 $i \times 2 + 1$ 。



堆的逻辑结构



堆的存储结构

存储固然简单，但是题目不会那么好心，直接给出一个标准的堆。

一般给我们的是一组数，这组数按照给出的顺序还原成二叉树之后，并不是一个堆结构。此时如果想将这组数变成堆的话，有两种操作：

1. 用数组存下来这组数，然后把数组调整成一个堆；
2. 创建一个堆，然后将这组数依次插入到堆中。

3. 核心操作

堆中的所有运算，比如建堆，向堆中插入元素以及删除元素等，都是基于堆中的两个核心操作实现的 ---- 向上调整算法以及向下调整算法。

因此，在实现堆之前，先来掌握两种核心操作。

注意：以下所有操作都默认堆是一个大根堆，小根堆的原理反着来即可。

3.1 向上调整算法

（可以用 ppt 展示）

算法流程：

1. 与父结点的权值作比较，如果比它大，就与父亲交换；
2. 交换完之后，重复 1 操作，直到比父亲小，或者换到根节点的位置。

代码实现：

```
int n; // 标记堆的大小
int heap[N]; // 存堆 - 默认是一大根堆

// 向上调整算法
void up(int child)
{
    int parent = child / 2;

    // 如果父结点存在，并且权值比父结点大
    while(parent >= 1 && heap[child] > heap[parent])
    {
        swap(heap[child], heap[parent]);
        // 交换之后，修改下次调整的父子关系，注意顺序不能颠倒
        child = parent;
        parent = child / 2;
    }
}
```

时间复杂度：

> 最差情况需要走一个树高，因此时间复杂度为 $\log N$ 。

3.2 向下调整算法

算法流程：

1. 找出左右儿子中权值最大的那个，如果比它小，就与其交换；
2. 交换完之后，重复 1 操作，直到比儿子结点的权值都大，或者换到叶节点的位置。

代码实现：

```
int n; // 标记堆的大小
int heap[N]; // 存堆 - 默认是一大根堆

// 向下调整算法
void down(int parent)
{
    int child = parent * 2;

    while(child <= n) // 如果还有孩子
    {
        // 找出两个孩子谁是最大的
        if(child + 1 <= n && heap[child + 1] > heap[child]) child++;
        // 最大的孩子都比我小，说明是一个合法的堆
        if(heap[child] <= heap[parent]) return;

        swap(heap[child], heap[parent]);
        // 交换之后，修改下次调整的父子关系，注意顺序不能颠倒
        parent = child;
        child = parent * 2;
    }
}
```

时间复杂度：

> 最差情况需要走一个树高，因此时间复杂度为 $\log N$ 。

4. 堆的模拟实现

> 掌握好向上调整算法以及向下调整算法，堆的实现就变得巨简单了。

（可以用 ppt 简单演示，然后直接写代码）

4.1 创建

> - 创建一个足够大的数组充当堆；
> - 创建一个变量 n，用来标记堆中元素的个数。

```
const int N = 1e6 + 10;

int n; // 标记堆的大小
int heap[N]; // 存堆 - 默认是一大根堆
```

4.2 插入

> 把新来的元素放在最后一个位置，然后从最后一个位置开始执行一次向上调整算法即可。

```
// 向上调整算法
void up(int child)
{
    int parent = child / 2;

    // 如果比父结点大
    while(parent >= 1 && heap[child] > heap[parent])
    {
        swap(heap[child], heap[parent]);
        child = parent;
        parent = child / 2;
    }
}

// 插入
void push(int x)
{
    // 把该元素放在最后
    heap[++n] = x;

    // 执行一次向上调整算法
```

```
    up(n);
}
```

时间复杂度：

> 时间开销在向上调整算法上，时间复杂度为 $O(\log N)$。

4.3 删除栈顶元素

> 1. 将栈顶元素和最后一个元素交换，然后 n--，删除最后一个元素；
> 2. 从根节点开始执行一次向下调整算法即可。

```
// 向下调整算法
void down(int parent)
{
    int child = parent * 2;

    while(child <= n) // 如果还有孩子
    {
        // 找出两个孩子谁是最大的
        if(child + 1 <= n && heap[child + 1] > heap[child]) child++;
        // 最大的孩子都比我小，说明是一个合法的堆
        if(heap[child] <= heap[parent]) return;

        swap(heap[child], heap[parent]);
        parent = child;
        child = parent * 2;
    }
}

// 删除
void pop()
{
    // 把第一个元素与最后一个元素交换
    swap(heap[1], heap[n]);
    n--;
    // 执行一次向下调整算法
    down(1);
}
```

时间复杂度：

> 时间开销在向上调整算法上，时间复杂度为 $O(\log N)$。

4.4 堆顶元素

- 下标为 1 位置的元素，就是堆顶元素。

```
1   // 堆顶元素
2   int top()
3   {
4       return heap[1];
5   }
```

时间复杂度：

> 显然是 $O(1)$。

4.5 堆的大小

- n 的值。

```
1   // 堆的大小
2   int size()
3   {
4       return n;
5   }
```

时间复杂度：

> 显然是 $O(1)$。

4.6 所有测试代码

```
1   #include <iostream>
2
```

```cpp
using namespace std;

const int N = 1e6 + 10;

int n;
int heap[N];

// 向上调整算法
void up(int child)
{
    int parent = child / 2;

    // 当父结点存在，并且大于父结点的权值时
    while(parent >= 1 && heap[child] > heap[parent])
    {
        swap(heap[child], heap[parent]);

        child = parent;
        parent = child / 2;
    }
}

// 向下调整算法
void down(int parent)
{
    int child = parent * 2;

    // 如果存在孩子
    while(child <= n)
    {
        // 找出最大的孩子
        if(child + 1 <= n && heap[child + 1] > heap[child]) child++;
        if(heap[parent] >= heap[child]) return;

        swap(heap[parent], heap[child]);
        parent = child;
        child = parent * 2;
    }
}

// 插入元素
void push(int x)
{
    heap[++n] = x;
    up(n);
}

```

```cpp
// 删除堆顶元素
void pop()
{
    swap(heap[1], heap[n]);
    n--;
    down(1);
}

// 查询堆顶元素
int top()
{
    return heap[1];
}

// 堆的大小
int size()
{
    return n;
}

int main()
{
    // 测试堆
    int a[10] = {1, 41, 23, 10, 11, 2, -1, 99, 14, 0};

    // 将这 10 个数依次放入堆中
    for(int i = 0; i < 10; i++)
    {
        push(a[i]);
    }

    while(size())
    {
        cout << top() << " ";
        pop();
    }

    return 0;
}
```

5. priority_queue

5.1 优先级队列

普通的队列是一种**先进先出**的数据结构，即元素插入在队尾，而元素删除在队头。

而在**优先级队列**中，元素被赋予优先级，当插入元素时，同样是在队尾，但是会根据**优先级进行位置调整**，优先级越高，调整后的位置越靠近队头；同样的，删除元素也是根据优先级进行，优先级最高的元素（队头）最先被删除。

其实可以认为，优先级队列就是堆实现的一个数据结构。

priority_queue 就是 C++ 提供的，已经实现好的优先级队列，底层实现就是一个**堆**结构。在算法竞赛中，如果是需要使用堆的题目，一般就直接用现成的 priority_queue，很少手写一个堆，因为省事~

5.2 创建 priority_queue - 初阶

> 优先级队列的创建结果有很多种，因为需要**根据实际需求**，可能会创建出来**各种各样**的堆：
>
> 1. 简单**内置类型**的大根堆或小根堆：比如存储 `int` 类型的大根堆或小根堆；
> 2. 存储**字符串**的大根堆或小根堆；
> 3. 存储**自定义类型**的大根堆或小根堆：比如堆里面的数据是一个**结构体**。
>
> 关于每一种创建结果，都需要有与之对应的写法。在初阶阶段，先用简单的 `int` 类型建堆，重点学习priority_queue 的用法。
>
> 注意：`priority_queue` 包含在 `queue` 这个头文件中。

```cpp
#include <iostream>
#include <vector>
#include <queue> // 优先级队列的头文件在 queue 里面

using namespace std;

// 优先级队列的使用
void test1()
{
    int a[10] = {1, 41, 23, 10, 11, 2, -1, 99, 14, 0};

    priority_queue<int> heap; // 默认写法下，是一个大根堆
}
```

5.3 size / empty

> 1. `size`：返回元素的个数。
> 2. `empty`：返回优先级队列是否为空。

> 时间复杂度：$O(1)$。

5.4 push

- 往优先级队列里面添加一个元素。

> 时间复杂度：因为底层是一个堆结构，所以时间复杂度为 $O(\log N)$。

5.5 pop

- 删除优先级最高的元素。

> 时间复杂度：因为底层是一个堆结构，所以时间复杂度为 $O(\log N)$。

5.6 top

- 获取优先级最高的元素。

> 时间复杂度：$O(1)$。

5.7 所有测试代码

```cpp
#include <iostream>
#include <vector>
#include <queue> // 优先级队列的头文件在 queue 里面

using namespace std;

// 优先级队列的使用
void test()
{
    int a[10] = {1, 41, 23, 10, 11, 2, -1, 99, 14, 0};

    priority_queue<int> heap; // 默认写法下，是一个大根堆

    for(auto x : a)
    {
        heap.push(x); // 插入元素
    }

    while(heap.size())
    {
```

```
        cout << heap.top() << " "; // 获取堆顶元素的值
        heap.pop(); // 删除元素
    }
}

int main()
{
    test();

    return 0;
}
```

5.8 创建 priority_queue - 进阶

5.8.1 内置类型

内置类型就是 C++ 提供的数据类型，比如 `int、double、long long` 等。以 `int` 类型为例，分别创建大根堆和小根堆。

```
#include <iostream>
#include <vector>
#include <queue> // 优先级队列的头文件在 queue 里面

using namespace std;

// 内置类型
void test()
{
    int a[10] = {1, 41, 23, 10, 11, 2, -1, 99, 14, 0};

    // 大根堆
    priority_queue<int> heap1; // 默认就是大根堆
    // priority_queue<数据类型， 存数据的结构， 数据之间的比较方式>
    priority_queue<int, vector<int>, less<int>> heap2; // 也是大根堆

    // 小根堆
    priority_queue<int, vector<int>, greater<int>> heap3; // 小根堆

    // 测试
    for(auto x : a)
    {
        heap1.push(x);
        heap2.push(x);
```

```
25          heap3.push(x);
26      }
27
28      while(heap1.size())
29      {
30          cout << heap1.top() << " "; // 获取堆顶元素的值
31          heap1.pop(); // 删除元素
32      }
33      cout << endl;
34
35      while(heap2.size())
36      {
37          cout << heap2.top() << " "; // 获取堆顶元素的值
38          heap2.pop(); // 删除元素
39      }
40      cout << endl;
41
42      while(heap3.size())
43      {
44          cout << heap3.top() << " "; // 获取堆顶元素的值
45          heap3.pop(); // 删除元素
46      }
47      cout << endl;
48  }
49
50  int main()
51  {
52      test();
53
54      return 0;
55  }
```

5.8.2 结构体类型

当优先级队列里面存的是结构体类型时，需要在**结构体中重载 `<` 比较运算符**，从而创建出大根堆或者小根堆。

> 注：这里的主要目的是把优先级队列创建出来，因此只用掌握写法即可。关于其中背后的原理，这里就不再赘述，因为涉及比较复杂的 C++ 语法知识。在这里，学习的侧重点就是模仿。

```
1   #include <iostream>
2   #include <vector>
3   #include <queue> // 优先级队列的头文件在 queue 里面
4
```

```cpp
using namespace std;

struct node
{
    int a, b, c;

    // // 以 b 为基准，定义大根堆
    // bool operator < (const node& x) const
    // {
    //     return b < x.b;
    // }

    // 以 b 为基准，定义小根堆
    bool operator < (const node& x) const
    {
        return b > x.b;
    }

};

void test()
{
    priority_queue<node> heap;

    for(int i = 1; i <= 10; i++)
    {
        heap.push({i, i + 1, i + 2});
    }

    while(heap.size())
    {
        auto [a, b, c] = heap.top();
        heap.pop();
        cout << a << " " << b << " " << c << endl;
    }
}

int main()
{
    test();

    return 0;
}
```

6. 堆的算法题

1. 【模版】堆

> **题目来源：** 洛谷
>
> **题目链接：** P3378 【模板】堆
>
> **难度系数：** ★

【题目描述】

> 给定一个数列，初始为空，请支持下面三种操作：
>
> 1. 给定一个整数 x ，请将 x 加入到数列中。
> 2. 输出数列中最小的数。
> 3. 删除数列中最小的数（如果有多个数最小，只删除 1 个）。

【输入描述】

> 第一行是一个整数，表示操作的次数 n 。
> 接下来 n 行，每行表示一次操作。每行首先有一个整数 op 表示操作类型。
>
> - 若 $op = 1$ ，则后面有一个整数 x ，表示要将 x 加入数列。
> - 若 $op = 2$ ，则表示要求输出数列中的最小数。
> - 若 $op = 3$ ，则表示删除数列中的最小数。如果有多个数最小，只删除 1 个。
>
> 对于 100% 的数据，保证 $1 \leq n \leq 10^6$ ， $1 \leq x < 2^{31}$ ， $op \in \{1, 2, 3\}$ 。

【输出描述】

> 对于每个操作 2 ，输出一行一个整数表示答案。

【示例一】

> 输入：
>
> ```
> 5
> 1 2
> 1 5
> 2
> 3
> 2
> ```
>
> 输出：
>
> ```
> 2
> ```

```
5
```

【解法】

不要偷懒，手写一个小根堆~

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int n; // 标记堆的大小
int heap[N]; // 小根堆

// 向上调整算法
void up(int child)
{
    int parent = child / 2;

    while(parent >= 1 && heap[child] < heap[parent])
    {
        swap(heap[child], heap[parent]);

        child = parent;
        parent = child / 2;
    }
}

// 向下调整算法
void down(int parent)
{
    int child = parent * 2; // 左孩子

    while(child <= n)
    {
        // 找出两个孩子中的最小值
        if(child + 1 <= n && heap[child + 1] < heap[child]) child++;

        if(heap[child] >= heap[parent]) return;

        swap(heap[child], heap[parent]);
        parent = child;
        child = parent * 2;
```

```cpp
    }
}

// 进堆
void push(int x)
{
    n++;
    heap[n] = x;

    up(n);
}

// 删除堆顶元素
void pop()
{
    swap(heap[1], heap[n]);
    n--;

    down(1);
}

int main()
{
    int T; cin >> T;
    while(T--)
    {
        int op; cin >> op;

        if(op == 1) // 进堆
        {
            int x; cin >> x;
            push(x);
        }
        else if(op == 2) // 输出堆顶
        {
            cout << heap[1] << endl;
        }
        else // 删除堆顶元素
        {
            pop();
        }
    }

    return 0;
}
```

2. 第 k 小

> **题目来源：** 牛客网
>
> **题目链接：** 第 k 小
>
> **难度系数：** ★

【题目描述】

> 有一个长度为 n 的数组，值为 $a[i]$, 牛牛想找到数组中第 k 小的数。比如 $[1,2,3,4,5,6]$ 中，第 3 小的数就是 2 。
>
> 牛牛觉得这个游戏太简单了，想加一点难度，现在牛牛有 m 个操作，每个操作有两种类型。
>
> `1 x`： 1 代表操作一，给数组中加一个元素 x 。$(0 \leq x \leq 1e9)$
>
> `2`：代表操作二，查询第 k 小的数。如果没有 k 个数就输出 -1 。

【输入描述】

> 第一行有三个整数， $n,m,k(1 \leq n,m,k \leq 2e5)$;
>
> 第二行包含 n 个整数 $a[i](0 \leq a[i] \leq 1e9)$;
>
> 接下来 m 行，每行代表一个操作。具体见题目描述。

【输出描述】

> 每次查询输出一个第 k 小的数。

【示例一】

> 输入：
>
> ```
> 5 4 3
> 1 2 3 4 5
> 2
> 1 1
> 1 3
> 2
> ```
>
> 输出：
>
> ```
> 3
> 2
> ```

【解法】

堆的经典应用 --- topK 问题。

如果是找第 k 小，解决方式：

1. 维护一个大小为 k 的大根堆；
2. 对于每次来的元素，先进堆，然后删除堆顶元素，此时堆顶元素就是第 k 小。

【参考代码】

```cpp
#include <iostream>
#include <queue>

using namespace std;

int n, m, k;
priority_queue<int> heap; // 默认就是大根堆

int main()
{
    cin >> n >> m >> k;

    for(int i = 1; i <= n; i++)
    {
        int x; cin >> x;
        heap.push(x);
        if(heap.size() > k) heap.pop();
    }

    while(m--)
    {
        int op; cin >> op;
        if(op == 1) // 来了一个数
        {
            int x; cin >> x;
            heap.push(x);
            if(heap.size() > k) heap.pop();
        }
        else if(op == 2) // 查询第 k 小
        {
            if(heap.size() == k) cout << heap.top() << endl;
            else cout << -1 << endl;
        }
    }

    return 0;
}
```

3. 除2!

> **题目来源：** 牛客网
>
> **题目链接：** 除2!
>
> **难度系数：** ★

【题目描述】

> 给一个数组，一共有 n 个数。
>
> 你能进行最多 k 次操作。每次操作可以进行以下步骤：
>
> - 选择数组中的一个偶数 a_i，将其变成 $a_i/2$。
>
> 现在你进行不超过 k 次操作后，让数组中所有数之和尽可能小。请输出这个最小的和。

【输入描述】

> 第一行输入两个正整数 n 和 k，用空格隔开
>
> 第二行输入 n 个正整数 a_i
>
> 数据范围：
>
> $1 \leq n \leq 100000$，$1 \leq k \leq 10^9$
>
> $1 \leq a_i \leq 10^9$

【输出描述】

> 一个正整数，代表和的最小值。

【示例一】

> 输入：
>
> 5 3
>
> 2 4 8 10 11
>
> 输出：
>
> 24

【解法】

搞一个堆模拟一下就好了~

【参考代码】

```
#include <iostream>
#include <queue>
```

```cpp

using namespace std;

typedef long long LL;

int n, k;
priority_queue<int> heap; // 默认就是大根堆

int main()
{
    cin >> n >> k;
    LL sum = 0; // 标记所有元素的和

    for(int i = 1; i <= n; i++)
    {
        int x; cin >> x;
        sum += x;
        if(x % 2 == 0) heap.push(x);
    }

    while(heap.size() && k--)
    {
        int t = heap.top() / 2;
        heap.pop();

        sum -= t;
        if(t % 2 == 0) heap.push(t);
    }

    cout << sum << endl;

    return 0;
}
```

4. 最小函数值

> **题目来源：** 洛谷
>
> **题目链接：** P2085 最小函数值
>
> **难度系数：** ★★

【题目描述】

> 有 n 个函数，分别为 $F_1, F_2, \ldots, F_n$。定义 $F_i(x) = A_i x^2 + B_i x + C_i (x \in N^*)$。给定这些 A_i、B_i 和 C_i，请求出所有函数的所有函数值中最小的 m 个（如有重复的要输出多个）。

【输入描述】

> 第一行输入两个正整数 n 和 m。
>
> 以下 n 行每行三个正整数，其中第 i 行的三个数分别为 A_i、B_i 和 C_i。
>
> 对于全部的测试点，保证 $1 \leq n, m \leq 10000$，$1 \leq A_i \leq 10, B_i \leq 100, C_i \leq 10^4$。

【输出描述】

> 输出将这 n 个函数所有可以生成的函数值排序后的前 m 个元素。这 m 个数应该输出到一行，用空格隔开。

【示例一】

> 输入：
>
> 3 10
>
> 4 5 3
>
> 3 4 5
>
> 1 7 1
>
> 输出：
>
> 9 12 12 19 25 29 31 44 45 54

【解法】

暴力解法：直接把所有函数代入 $[1, m]$ 的结果全部算出来放进小根堆中，然后依次取出 m 个堆顶元素即可。

但是时间复杂度为 $O(N \times M)$，会超时。依旧利用堆，但是要转化一下思想。

由题意可知，所有的二次函数的 A 和 B 都是大于 0 的，因此在 $x > 0$ 的区间内，所有函数值都是递增的。也就是说，如果某个函数值 $F_i[x]$ 不是堆顶元素，那么 $F_i[x+1]$ 也必定不是堆顶元素。

由此可见，大可不必把所有的结果都算出来之后再放进去。可以先把所有函数代入 1 的结果放进堆里面。然后每次取出堆顶元素之后，就把堆顶元素对应函数的下一个结果算出来放进去。因为所有的函数都是递增的，因此这种策略绝对是正确的。

由于取出堆顶元素之后要确定当前堆顶元素是谁，并且代入的数是多少。所以堆里面需要存一个结构体，标注：函数值 + 哪个函数 + 代入的值，然后按照函数值创建小根堆即可。

【参考代码】

```cpp
#include <iostream>
#include <queue>

using namespace std;

typedef long long LL;
const int N = 1e4 + 10;

int n, m;
LL a[N], b[N], c[N];

struct node
{
    LL f; // 函数值
    LL num; // 函数编号
    LL x; // 代入值

    bool operator <(const node& x) const
    {
        // 小根堆，大元素下坠
        return f > x.f;
    }
};

priority_queue<node> heap;

LL calc(LL i, LL x)
{
    return a[i] * x * x + b[i] * x + c[i];
}

int main()
{
    cin >> n >> m;
    for(int i = 1; i <= n; i++)
    {
        cin >> a[i] >> b[i] >> c[i];
    }

    // 1. 把 x = 1 的值放入堆中
    for(int i = 1; i <= n; i++)
    {
        heap.push({calc(i, 1), i, 1});
    }

    // 2. 依次拿出 m 个值
    while(m--)
```

```
    {
        auto t = heap.top(); heap.pop();
        LL f = t.f, num = t.num, x = t.x;

        cout << f << " ";
        // 把下一个函数值放入堆中
        heap.push({calc(num, x + 1), num, x + 1});
    }


    return 0;
}
```

5. 序列合并

题目来源： 洛谷

题目链接： P1631 序列合并

难度系数： ★★

【题目描述】

有两个长度为 N 的单调不降序列 A, B，在 A, B 中各取一个数相加可以得到 N^2 个和，求这 N^2 个和中最小的 N 个。

【输入描述】

第一行一个正整数 N；

第二行 N 个整数 $A_{1...N}$。

第三行 N 个整数 $B_{1...N}$。

对于 100% 的数据，$1 \leq N \leq 10^5$，$1 \leq a_i, b_i \leq 10^9$。

【输出描述】

一行 N 个整数，从小到大表示这 N 个最小的和。

【示例一】

输入：

```
3
2 6 6
1 4 8
```

> 输出：
>
> 3 6 7

【解法】

本质的算法原理和《最小函数值》一致。

当两个序列是单调不降时，有如下单调性：

- $a[1]+b[1] \le a[1]+b[2] \le a[1]+b[3] \le ... \le a[1]+b[n]$;
- $a[2]+b[1] \le a[2]+b[2] \le a[2]+b[3] \le ... \le a[2]+b[n]$;
- ....

因此，可以创建一个小根堆，并把 $a[1...n]+b[1]$ 全部放进去，然后每次拿出堆顶元素 $a[i]+b[j]$ 时，再把 $a[i]+b[j+1]$ 放进去，这样重复 n 次即可 。

【参考代码】

```cpp
#include <iostream>
#include <queue>

using namespace std;

const int N = 1e5 + 10;

int n;
int a[N], b[N];

struct node
{
    int sum; // 当前的和
    int i, j; // a 和 b 的编号

    bool operator < (const node& x) const
    {
        // 小根堆，大元素下坠
        return sum > x.sum;
    }
};

priority_queue<node> heap; // 小根堆

int main()
{
```

```
27      cin >> n;
28      for(int i = 1; i <= n; i++) cin >> a[i];
29      for(int i = 1; i <= n; i++) cin >> b[i];
30
31      // 1. 将 a[i] + b[1] 放进堆里面
32      for(int i = 1; i <= n; i++)
33      {
34          heap.push({a[i] + b[1], i, 1});
35      }
36
37      // 2. 依次拿出 n 的数
38      for(int k = 1; k <= n; k++)
39      {
40          node t = heap.top(); heap.pop();
41          int sum = t.sum, i = t.i, j = t.j;
42
43          cout << sum << " ";
44          if(j + 1 <= n) heap.push({a[i] + b[j + 1], i, j + 1});
45      }
46
47      return 0;
48  }
```

6. 舞蹈课

> **题目来源：** 洛谷
>
> **题目链接：** P1878 舞蹈课
>
> **难度系数：** ★★★

【题目描述】

> 有 n 个人参加一个舞蹈课。每个人的舞蹈技术由整数来决定。在舞蹈课的开始，他们从左到右站成一排。当这一排中至少有一对相邻的异性时，舞蹈技术相差最小的那一对会出列并开始跳舞。如果不止一对，那么最左边的那一对出列。一对异性出列之后，队伍中的空白按原顺序补上（即：若队伍为 `ABCD`，那么 `BC` 出列之后队伍变为 `AD`）。舞蹈技术相差最小即是 a_i 的绝对值最小。
>
> 任务是模拟以上过程，确定跳舞的配对及顺序。

【输入描述】

> 第一行一个正整数 n 表示队伍中的人数。
>
> 第二行包含 n 个字符 `B` 或者 `G`，`B` 代表男，`G` 代表女。
>
> 第三行为 n 个整数 a_i。所有信息按照从左到右的顺序给出。

【输出描述】

> 第一行一个整数表示出列的总对数 k 。
>
> 接下来 k 行，每行是两个整数。按跳舞顺序输出，两个整数代表这一对舞伴的编号（按输入顺序从左往右 1 至 n 编号）。请先输出较小的整数，再输出较大的整数。

【示例一】

> 输入：
>
> 4
>
> BGBG
>
> 4 2 4 3
>
> 输出：
>
> 2
>
> 3 4
>
> 1 2

【解法】

首先，要经常拿到异性中舞蹈技术差值最小的值，所以考虑到用堆来解决问题，也就是用堆存所有相邻的异性，然后根据技术差建立小根堆。接下来就是处理一些难题和细节问题：

- 一对相邻的异性出列之后，他们左右会变成相邻，如何处理？

  这个操作相当于在一个序列中删除两个相邻元素，删除之后不影响剩下元素的先后顺序，最快的实现方式就是双向链表。

  因此，可以用双向链表存储所有数据，当删除两个相邻异性之后，仅需修改指针即可。并且能快速找到删除之后，新接触的两个人的位置，方便判断是否是未出列的异性，然后加入到堆中。

- 当相邻异性的技术差相等的时候，需要优先输出靠前位置的一对，如何处理？

  堆中存一个结构体，这个结构体中包含这对异性的：左编号，右编号以及技术差。然后重载 `<` 运算符时，特殊处理差值相等的情况：

  - 如果技术差不等：按照技术差建立小根堆；
  - 如果技术差相等，左编号不等：按照左编号建立小根堆；
  - 如果技术差相等，左编号相等：按照右编号建立小根堆。

- 堆中可能存在已经出列的人，怎么处理？

  创建一个 `bool` 数组标记一下哪些人已经出列。接下来从堆中拿出一对异性时，就先判断是否已经出过列了。

【参考代码】

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <cmath>

using namespace std;

const int N = 2e5 + 10;

int n;
int s[N]; // 标记男女 - 0/1

// 双向链表存数据
int e[N];
int pre[N], ne[N];

struct node
{
    int d; // 技术差
    int l, r; // 左右编号

    // 小根堆，大元素下坠
    bool operator < (const node& x) const
    {
        if(d != x.d) return d > x.d;
        else if(l != x.l) return l > x.l;
        else return r > x.r;
    }
};

priority_queue<node> heap;
bool st[N]; // 标记已经出队的人

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        char ch; cin >> ch;
        if(ch == 'B') s[i] = 1;
    }

    for(int i = 1; i <= n; i++)
    {
        cin >> e[i];
        // 创建双向链表
```

```
47          pre[i] = i - 1;
48          ne[i] = i + 1;
49      }
50      pre[1] = ne[n] = 0; // 0 表示后面没有元素
51
52      // 1. 先把所有的异性差放进堆中
53      for(int i = 2; i <= n; i++)
54      {
55          if(s[i] != s[i - 1])
56          {
57              heap.push({abs(e[i] - e[i - 1]), i - 1, i});
58          }
59      }
60
61      // 2. 提取结果
62      vector<node> ret; // 暂存结果
63
64      while(heap.size())
65      {
66          node t = heap.top(); heap.pop();
67          int d = t.d, l = t.l, r = t.r;
68
69          if(st[l] || st[r]) continue;
70
71          ret.push_back(t);
72          st[l] = st[r] = true; // 标记 l 和 r 已经出列
73
74          // 修改指针，还原新的队列
75          ne[pre[l]] = ne[r];
76          pre[ne[r]] = pre[l];
77
78          // 判断新的左右是否会成为一对
79          int left = pre[l], right = ne[r];
80          if(left && right && s[left] != s[right])
81          {
82              heap.push({abs(e[left] - e[right]), left, right});
83          }
84      }
85
86      cout << ret.size() << endl;
87      for(auto& x : ret)
88      {
89          cout << x.l << " " << x.r << endl;
90      }
91
92
93      return 0;
```

```
94    }
```